CSEN 177: Operating Systems
Lab 5: Synchronization using semaphores, locks, and condition variables

Objectives

1. To use semaphores, locks, and condition variables for synchronization
2. To develop a C program to coordinate the access of producer and consumer threads to a shared buffer

Guidelines

You have learned in class that threads run concurrently, and when they read/write to a shared memory, the program behavior is undefined. This is because the CPU scheduler switches rapidly between threads to provide concurrent execution. One thread may only partially complete execution before another thread is scheduled. Therefore, a thread may be interrupted at any point in its instruction stream, and the CPU may be assigned to execute instructions of another thread, so the thread schedule is non-deterministic, and the resulting output is non-reproducible. Synchronization is required to control the non-deterministic and non-reproducible behavior of multi-threaded programs.

Each thread has a segment of code that involves data sharing with one or more threads. This code segment is referred to as a critical section. Synchronization imposes a rule that when one thread executes in its critical section, no other thread can execute in its critical section. Each thread must request permission to enter its critical section, formally defined as the entry section. When a thread completes execution in the critical section, it leaves through an exit section to the remaining code of the program. The general structure of synchronization is therefore defined as follows:

```
do {
   entry section
      critical section
   exit section
   remainder section
  } while (1);
```

A variety of synchronization tools exit. This lab uses semaphores, mutex locks, and condition variables for demonstration. A semaphore is considered a generalized lock, and it supports two operations:

- P(): an atomic operation that waits for the semaphore to become positive, then decrements it by 1. This operation is referred to as wait() operation
- V(): an atomic operation that increments the semaphore by 1, waking up a waiting P, if any. This operation is referred to as signal() operation.

P() stands for “proberen” (to test) and V() stands for “verhogen” (to increment) in Dutch. Linux provides a high-level APIs for semaphores in the <semaphore.h> library:

```
sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);
```

Note: MacOs does not support sem_init and sem_destroy (unnamed semaphores). If you are using MacOS, use a named semaphore with sem_open, and sem_unlink as follows:

```
sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);
int sem_unlink(const char *name);
```

Mutex lock is a synchronization variable, if one thread holds, no other thread can hold it. A lock has two operations: lock (acquire) and unlock (release), Linux provides a high-level API for condition variables in the <pthread.h> library:

```
pthread_mutex_t lock; //Declare a lock
pthread_mutex_init(&lock, NULL); //Create a lock
pthread_mutex_lock(&lock); //lock acquire
pthread_mutex_unlock(&lock); //lock release
pthread_mutex_destroy(&mutex); // delete lock
```

Condition variables provide another way for threads to synchronize. They allow threads to synchronize based upon the actual value of data (note: mutex implements synchronization by controlling thread access to data)

A condition variable is a synchronization object that lets a thread efficiently wait for a change to a shared state protected by a lock. A condition variable is designed to work in conjunction with locks. Linux provides a high-level API for condition variables in the <pthread.h> library:

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
```

C Program with semaphores

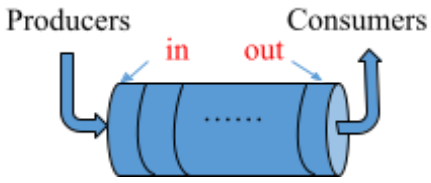
In lab 4, you demonstrated the use of threads and multi-threaded programs that do not have dependencies. In this lab, you will use semaphores to illustrate the use of dependent threads. Demonstrate each of the following steps to the TA to get a grade on this part of the lab assignment.

Step 1. [15%] Download the threadSync.c program from Camino, then compile and run it several times. Explain what happens when you run the program and how it differs from the thread programs you used in Lab 4.

Step 2. [25%] Modify threadSync.c in Step 1using mutex Locks.

Producer – Consumer as a classical problem of synchronization

Step 3. [30%] The following diagram illustrates the bounded buffer consumer-producer problem. A consumer can consume only when the number of (consumable) items is at least 1. A producer can produce only when the number of empty spaces is at least 1. The index to point to a produced item is in, and the index to point to a consumed item is out. Recall that the buffer is a shared variable and is accessible between producers and consumers.



Bounded buffer consumer-producer

Write a program illustrating producer and consumer threads coordinating to access the shared buffer using semaphores. You may use the following pseudo-code.

```
//Shared data: semaphore full, empty, mutex;
//pool of n buffers, each can hold one item
//mutex provides mutual exclusion to the buffer pool
//empty and full count the number of empty and full buffers
//Initially: full = 0, empty = n, mutex = 1

//Producer thread
do {
   …
   produce next item
    …
   wait(empty);
   wait(mutex);
    …
   add the item to buffer
    …
   signal(mutex);
   signal(full);
} while (1);

//Consumer thread
do {
   wait(full)
```

```
    wait(mutex);
     …
    remove next item from buffer
     …
    signal(mutex);
    signal(empty);
     …
    consume the item
     …
} while (1);
```

Step 4. [30%] When employing mutex locks to coordinate access to a shared buffer between producers and consumers, the issue of busy waiting may arise. To address this, a condition variable is utilized as an explicit queue where threads can place themselves when certain conditions of execution are not met. This allows threads to wait on the condition, thereby avoiding busy waiting. A lock is employed to facilitate one thread, signaling to others that they may proceed if they are waiting for a particular condition to be met. Write a C program to illustrate the use of condition variables for coordinating the access of the producer and consumer threads to the shared buffer. You may use the following pseudo-code for implementation.

```
//Producer thread
do {
   …
   produce next item
    …
   lock(mutex);
   while (buffer is full)
          condV.wait(empty, mutex);
    …
   add the item to buffer
    …
   condV.signal(full);
   unlock(mutex);
} while (1);

//Consumer thread
do {
   lock(mutex)
   while (buffer is empty)
          condV.wait(full, mutex)
    …
   remove next item from buffer
    …
   condV.signal(empty);
   unlock(mutex);
    …
   consume the item
    …
} while (1);
```

Requirements to complete the lab

1. Show the TA the correct execution of the C programs.
2. Submit your answers to questions, observations, and notes as .txt file and upload to Camino
3. Submit the source code for all your programs as .c file(s) and upload to Camino.

Note: Be sure to retain copies of your .c and .txt files. You will want these for study purposes and to resolve any grading questions

Please start each program/ text with a descriptive block that includes the following information:

```
# Name: <your name>
# Date: <date> (the day you have lab)
```

```
# Title: Lab5 – task
# Description: This program computes … <you should
# complete an appropriate description here.>
```